

COMP 452 Assignment 3, Part 2 (Q-Learning) Description File

Bryce Dixon (3397649) Feb 2026

TABLE OF CONTENTS

- PART 1: Logic and Game Design - Page 1
- PART 2: Game Compilation Instructions - Page 10
- PART 3: Game Play Instructions - Page 11
- PART 4: Current Game Bugs/ Problems - Page 12

PART 1: Logic and Game Design

GAME CREATION ITERATIONS:

- I have never tried reinforcement learning before, so I knew I wanted to do a game with it.
- I initially set out to build a game where the AI learned to chase the user as the user moved around, and I then wanted to implement this into a hide and seek/ tag game. This led to me understanding the limitations of using a q-table, as the state space in this game design was very large and the training would take an incredible amount of time and creativity.
 - This state space was roughly:
 - 4 wall booleans, dx and dy to indicate direction and rough distance (close, near, far) to player (ranged from -3 to 3), pathBlocked boolean (to indicate if the path to the user was blocked by a wall to avoid the AI getting stuck trying to drive through the wall).
 - $4^2 \times 7 \times 7 \times 2 = 1568$ possible states
 - Due to the random and variable nature of the game, nearly all states were required for the AI to work correctly. However, training the agent became a difficult problem in itself, as to be able to train it in a manner that it experienced these states and did so consistently enough to learn the correct actions of these states.
 - After 3 days of modifying the Q-Learning and state parameters, trying exceptionally long training times, and modifying anything I could find or think of, I humbly accepted that my design was maybe too ambitious, and that I needed to scale back.
- This lead me to create this current game, where the AI agent learns to traverse a map and use actions, and then the user gets to race the AI agent through the map. This has a much more approachable amount of states and learns rather quickly.
 - The state space in my second game is:
 - 4 Wall booleans, 4 directions, breakable wall boolean
 - $4^2 \times 4 \times 2 = 128$ possible states
 - Of which, my game design typically uses 30-40 (according to the size of the Q-Table).

CLASS STRUCTURE OVERVIEW:

- Main
 - MapCreator

- MapCell
- ScreenDisplay
- GameManager
- GamePlayer
 - UserPlayer
 - ComputerPlayer
 - State
 - QLearner
 - **BreadthFirstSearch**
 - MapPoint

****NOTE: Breadth First Search** is used for reward shaping during Q-Learning, it is not used for pathfinding in the traditional sense. This is described in detail on pages 7 and 8 below.**

HIGH-LEVEL GAME LOGIC OVERVIEW:

- This is a competitive race game where the user races an AI agent through a map.
- The AI agent is trained on the map using q-learning (storing states in a q-table), and uses its state table to traverse the map afterwards.
- The map is navigated using up, down, left, right, and there are also "brick" walls, which the user presses space bar to break through. The AI agent learns to break through the wall to get through as well.
- The game detects when both players have finished the race and indicates who won.

GAME STATES LOGIC (*not q-learning states*)

- The game loop (in main.java) uses "game states" to determine what it is running.
- GameManager creates and holds 4 game states: Learning, Paused, Playing, Gameover. These game states are held within an enum object.
 - These states are set at various times so that the main gameloop is running the correct items and the ScreenDisplay.java is displaying the correct game screen to the user for each state.
- These 4 game states do the following:
 - **LEARNING:**
 - This is running the q learning algorithm for the AI agent to learn the map. This is displayed on the game map, and a countdown timer shows how much of the learning is left to complete.
 - **PAUSED:**
 - This is to indicate to the user that learning is complete, show the game instructions, and allow the user to press play when ready to play.
 - **PLAYING:**
 - When the game begins, the AI will start travelling through the maze.
 - The user can control their character through the map.
 - An output at the bottom of the screen displays the q-values for the AI agent's state. This was added so that the marker of this assignment could see some of the AI logic in real-time.

- GAMEOVER:
 - This runs when both players have finished the map.
 - If only one player has finished the map, the game freezes that player (stops updating it) and waits for the other player to finish before displaying the game over screen.
 - This screen outputs to the user who won the race.

USER LOGIC

- During the Learning, Paused, and Gameover game states, the user is unable to move their character.
- The user's key presses are held in a set, to allow for multiple key presses to be detected. This allows for the user to move in all directions as well as diagonally.
- Wall collisions are handled by getting the user's object's center position, and accounting for the radius of the object to determine the min and max rows and columns the object could be in. The program loops through these rows and columns, and checks if any are walls. If they are, the user is returned to their previous position and their velocity is zeroed out.
 - I initially tried a version of AABB circle-rectangle collision detection physics, but this was not working for this game variation, so I went with a simpler design, which turned out great.

COMPUTER AI AGENT LOGIC

- During the Learning game-state, the AI agent's q-learning algorithm is run to allow the user to learn the game map. I will describe this below:
- Q-LEARNING LOGIC:
 - The q-learning algorithm uses a Q table (a HashMap) which stores game states (the key) and their respective q values for each action of that state (the value is a double array of size 5).
 - States were built with 2 main ideas in mind:
 - 1. The AI Agent can only learn from information if it is readily available in the environment or in the state.
 - 2. Keeping state designs minimal will reduce the state space, and make learning faster and thorough.
 - Q Learning states included the following variables and actions:
 - VARIABLES (5): A. Wall Left, Right, Up, Down, and B. Direction, and C. Breakable Wall Nearby.
 - A. Wall Left, Right, Up, and Down indicated simply if there is a wall (breakable or regular walls included) in any of these directions. This was included as avoiding walls is crucial for maze tracking.
 - B. Direction indicates which direction the AI Agent is travelling. This uses values obtained from BreadthFirstSearch (discussed below), and was included as the same state with the same walls can exist but the

agent could be moving in either direction, so directionality was needed to break this state issue.

- The image below shows this clearly, as this was an issue I was running into initially.
- In Corridors A and B the agent has walls to the left and right. Once the agent moves through Corridor A, it would have learned that to navigate through the corridor, it needs to move up. However, once it arrives to corridor B, it will try this logic, and need to learn to move down, inevitably battling itself.
- Adding directionality allows the agent to be in a different state in both corridors, removing this oscillatory behaviour.

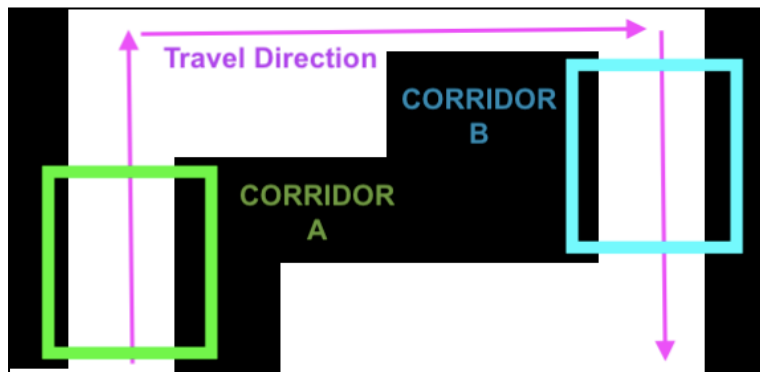


IMAGE 1: Visual Displaying Why Directionality is needed

- **C.** Breakable Wall Nearby simply indicates if the cells to the left or right of the agent are breakable brick walls.
 - This way the agent will learn that when this is present, that the break wall behaviour is used. This is described below.
- ACTIONS (5): Move Left, Right, Up, Down, or Break Brick.
 - Move Left, Right, Up, and Down move the agent to the next cell in any of these directions.
 - Break Brick executes an action which breaks breakable walls to the left or right of the agent.
- ON EACH LEARNING ITERATION the q-learning program gets:
 - The current state of the AI Agent.
 - The Q-values for that state. If the state is new, it initiates the q values for the state to 0 and adds the state to the table.
 - It then either:
 - Randomly selects an action if a randomly generated value is less than the epsilon value (epsilon is discussed below).
 - OR chooses the next best action based on the state's q values.
 - If there are multiple best actions, the program randomly selects one of them to avoid bias. This was added as the

first option (move left) was chosen very often and was an issue initially.

- Then it checks if the action causes the agent to hit a wall (breakable wall OR regular wall):
 - If it does → a negative reinforcement is given, the agent is kept in the same cell (not allowed to move into the wall), and the q-value is updated and this update loop iteration ends here.
 - If it does not → we execute the action (move one cell in the chosen direction OR break the brick action).
- It then checks:
 - If the agent found the finish line, it gets positive reinforcement, the agent is reset at the beginning, the q value is updated (and does not include a "next state" (discussed below)), and the iteration ends here.
 - If the agent broke a breakable wall when executing the break action, it gets positive reinforcement.
 - If the agent is moved further from the finish line it gets negative reinforcement, and positive reinforcement if it moved closer.
 - This is detected by comparing breadth first values of the cells (discussed below).
 - The q-value is then updated, the epsilon value is decayed, and this loop iteration ends.
- Q-LEARNING SPECIAL FEATURES
 - The total learning time is limited by a counter, which indicates to switch out of the learning state and into the paused state when it runs out.
 - When the algorithm selects a movement action, it moves an entire cell so that it is placed into another state. Initially this was not done (movement was incremental), which did not work well for learning (discussed more below).
 - When random actions are chosen, this does not include the "break wall" action so that learning movement is faster.
 - The break wall action is therefore learned when the agent comes to a breakable wall, is penalized for trying to move into any walls around it or moving further away, and eventually all other actions will be negatively reinforced, while break walls will be 0. It will then be selected and rewarded.
 - The agent is drawn towards the breakable wall as it is given a Bread First Search value, unlike regular walls (discussed below).
 - Once the wall is broken, the agent will be in a new state (as the breakable wall turns into an open space) and the agent continues moving.
 - When the q-value is updated, there are two possible formulas that are used:

- If the state reached is not a terminal state (i.e. there is a 'next state'), the q-value is updated using the Q-Learning rule as outlined in the course textbook.

$$Q(S, A) = (1 - \alpha)Q(S, A) + \alpha (reward + \gamma * (max(Q(S', A'))))$$

- Where $Q(S, A)$ is the q-value for the current state and action, α is the learning rate (discussed below), reward is the reinforcement value, γ is the discount factor (discussed below), and $max(Q(S', A'))$ is the next best action for the next state.
- Which I rearranged (steps shown below), as my q-values were not updating cumulatively, which seemed to be causing some issues, so I adjusted it to look similar to the formula in this reference example:

- <https://www.geeksforgeeks.org/machine-learning/q-learning-g-in-python>

Expand (1-alpha): $Q(S, A) = Q(S, A) - \alpha Q(S, A) + \alpha (reward + \gamma * (max(Q(S', A'))))$

Join common values: $Q(S, A) = Q(S, A) + \alpha * (reward + \gamma * (max(Q(S', A')) - Q(S, A))$

Above was coded as: $Q(S, A) += \alpha * (reward + \gamma * (max(Q(S', A')) - Q(S, A))$

- If the state reached is the terminal state, the q-value is updated without considering future learning (i.e. I removed $\gamma * (max(Q(S', A'))$ as my terminal state was not being definitively met with the learning. The agent was simply stopping just before it and hovering):

$$Q(S, A) += \alpha * (reward - Q(S, A))$$

- To avoid the agent being caught in a dead end (i.e. where all q-values begin stabilizing towards negative infinity), I put a maximum length for a learning episode so that the agent does not just keep hammering down the wrong path. If this timer is reached, the agent is placed back at the start position and continues from there.
- REINFORCEMENT VALUES USED:

SITUATION	VALUE	RATIONALE
Found finish line	+20	The finish line is the main goal so gets a large reward.
Broke breakable wall	+5	I wanted to ensure the agent learns this action clearly.
Moved closer to the finish	+2	I wanted to entice the agent for moving closer as it was getting stuck in its travels and having indecision.
Moved further from the finish	-1	I wanted to further strengthen the push to move closer to the finish, but not overpower anything so this value is lower.
Hit a wall	-5	If it hits a wall, I wanted this to be very negated so that the agent learns not to drive into walls.

If the action does none of the above	-0.02	This is to push the agent along in situations where none of the above items occur. This was seen in a lot of tutorials, although it is redundant in my design, I like the idea of ensuring the agent is pressured to reach a new state.
--------------------------------------	-------	---

■ Q-Learning algorithm parameters:

- The values for alpha, gamma, and epsilon were chosen based off of the example provided here:
 - <https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/>
- And then modified slightly with help from the descriptions found in this article:
 - <https://iopscience.iop.org/article/10.1088/1742-6596/1948/1/012038/pdf#:~:text=To%20overcome%20the%20two%20limitations,according%20to%20the%20experimental%20results.>
- Alpha (Learning rate) → value used: 0.2
 - As outlined in the course textbook, the alpha value determines how much we favour newly acquired information over older/ existing information.
 - For my game, I wanted to favour older information more, as it is a discrete, unchanging environment, as this also sped up the learning time.
- Gamma (Discount factor) → value used: 0.95
 - Also outlined in the textbook, the discount factor indicates how much a reward is valued. A higher value meant values obtained in the future are valued more, which made sense as I wanted my agent to value the rewards that came later on (like finding, or getting closer to, the finish and breaking brick walls).
- Epsilon (exploration factor) → value used: 0.3
 - This value is simply used to determine the amount of randomness in the agent's action selection. I wanted only a small amount of randomness as overall my agent had a much clearer direction (due to the breadth first search rewards used, which is discussed below). This value also decays slightly on every iteration to allow a reduction in randomness as learning progresses.

■ BREADTH FIRST SEARCH (BFS) USAGE:

- When building my game, I realized the agent would need something to incentivise it to move, or else learning would take incredibly long (and I did not want you to have to wait hours for the learning to complete). The agent had no way to know if it was

moving closer or further from the end of the map until it eventually reached the end of the map several times and this percolated through the q-values. Due to this, it would often get stuck on walls or corridors moving aimlessly back and forth.

- I decided a clean way to do this would be to run BFS from the finish line, and increment the value of each cell from the finish by 1. Only cells that will be traversed in 4 directions (left, right, up, and down, no diagonals) are looped through by the BFS algorithm, and all walls hold a value of infinity.
 - Except breakable walls, which are included in the BFS so that the agent is pulled towards them and can discover the break walls action, as described above.
- You will be able to see these BFS values during the learning phase.

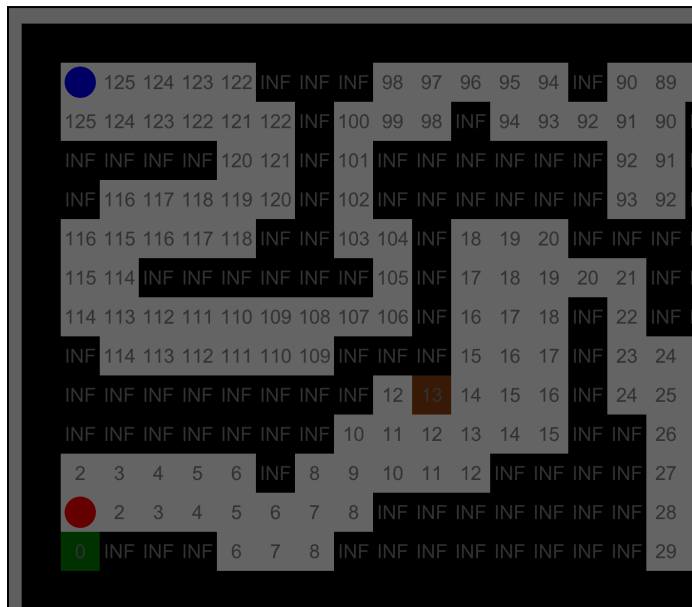


IMAGE 2: BFS values displayed during AI agent learning (note you can also see that black walls hold infinity and breakable (brown) walls include a BFS value)

- This results in the agent being able to know if it is moving in a good or bad direction, and in which direction it is moving:
 - REWARD SHAPING: If the action chosen moves them to a cell with a lower BFS value, they are moving closer to the finish, and are thus rewarded. If they are moving away from the finish, they are punished. If the BFS value is unchanged, they get a very small punishment.
 - DIRECTIONALITY: When at a given cell, the program checks the 4 cells around the agent, checks which has the lowest BFS value and indicates in which direction this cell

is. This tells us which direction the agent is (or should be) travelling.

- I was concerned that using BFS in this manner may "break" the rules a bit as this is notably shaping the rewards received, and may not be "true" AI learning. But I feel it is suitable for the problem to be solved efficiently and my AI is still using reinforcement to learn and using those states/ q- values to act in the environment.
- The BFS values are only used to indicate the direction the AI agent is moving in during gameplay. It is NOT used for path following in the sense that the agent doesn't just select the next best BFS node to move through the map.
- COMPUTER PLAYING/ MOVEMENT LOGIC
 - In order to use the q-table the agent built, the agent needed to move in a way that would utilize the states it found.
 - During learning, when the agent moves in any direction, it moved an entire cell width to get to the next cell, as incremental movement did not work for learning (the agent would get stuck (vibrate) in the cell it was in as all values would converge to the same value).
 - Thus during playing, when the agent gets a movement action, it sets the next cell in the direction of movement as the target, and the agent moves towards that target. Once it arrived there, it can check which state it is in, get the next action and repeat.
 - *You will see that the selection of the next cell is highlighted in **orange** during game play.*
 - This ensures the agent is in a consistent state during its movement through the map.
 - Up until the agent finishes the race, the update loop does the following:
 - Checks if the agent has arrived at its target cell.
 - If it has, it checks if it has finished the race.
 - If it has finished the race, the update loop ends here and the agent is no longer updated.
 - If it has NOT finished the race:
 - It gets its current state, selects the best action based on the q-values of this state (which is done in the same manner as described above for learning).
 - If it is a break wall action, it performs this, if not it sets the next target cell for the chosen movement action.
 - If it has not arrived at the target cell:
 - It executes its movement towards the target cell incrementally based on its speed and delta time.
 - It then updates the character's object position on the screen.

- It is important to note, the agent does not have any collision detection logic for avoiding walls or detecting breakable walls inherently. These features are all learned entirely within the Q-Learning steps and reward shaping.

PART 2: Game Compilation instructions

IF YOU ARE USING JAVA 8:

- Since JavaFX is included with Java8, you can simply navigate to the directory where you have saved the jar file and run:
 - `java -jar A3P2_452_Dixon_3397649.jar`

IF YOU ARE USING JAVA 11 and HIGHER:

- This project was developed and tested on a MacOS device using Java 17, meaning I needed to download and install JavaFX. I used JavaFX version 21.0.9.
- To run the .jar file, save the file to your device in a known location.
- Ensure JavaFX is downloaded, and find the path to the JavaFX 'lib' folder.
- In your command line interface (CLI), Navigate to the location where you saved the .jar file.
 - Now we tell java what libraries to include from JavaFX, as well as where to find them (i.e. the path to "lib" mentioned above), and then run the program.
 - An example of this is:


```
desktop % java \
--module-path /Users/bd/Desktop/AU/COMP_452/javafx/lib \
--add-modules javafx.controls,javafx.graphics \
-jar A3P2_452_Dixon_3397649.jar
```

 - Where the file (A3P2_452_Dixon_3397649.jar) is stored on on my desktop, the path to my javaFX library is `/Users/bd/Desktop/AU/COMP_452/javafx/lib`, and we are using the modules `javafx.controls,javafx.graphics` from the JavaFX library to run the program.

PART 3: Game Play Instructions

- The game rules are as follows:
 - Game objects (seen in image below):
 - The AI agent is the RED circle, you are the BLUE circle.
 - White spaces are open and drivable.
 - Black spaces are walls.
 - Brown spaces are breakable walls.
 - The green space at the end is the finish line.

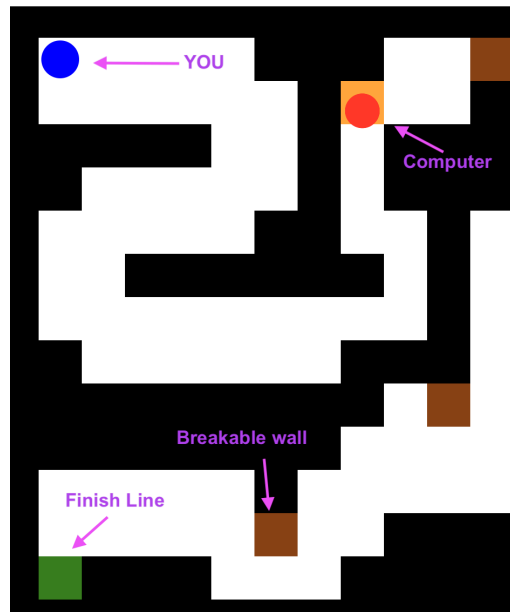


IMAGE 3: Game Objects

- While the AI agent is learning, you will not be able to move your character.
- After learning and once you press play:
 - Use the arrow keys to move your character.
 - You can move diagonally, but the computer cannot.
 - The computer moves faster but by moving diagonally, you can get through the map faster, which levels the playing field.
 - When you are beside a breakable wall, press the space bar.
 - You will see your character flash yellow when you do this (as seen below). You will also see the computer character do this as well.
 - This only works when you are in the cell to the left or right of the breakable wall.

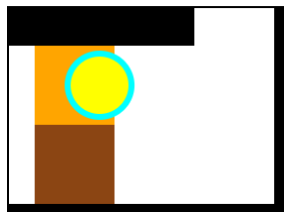


IMAGE 4: Character flashing yellow when pressing space bar to break through a breakable wall.

PART 4: Current Game Bugs/ Problems

- PROBLEMS/ LIMITATIONS
 - The game does not have much variability, as the agent's learning is very consistent, so it moves in the same manner on each iteration.
 - Once you figure out how to beat the computer, it is fairly easy to do so, although it is exciting for the first few times playing.
 - More AI agents could be added and modified in their learning to enhance gameplay.
 - Learning and the state of the agent rely heavily on the BFS algorithm for reward shaping. This provided a better result, and sped up learning, but may not be "true" learning.
 - Sound and sprites could be added to extend the game's appearance.
 - More features could be added to display the AI learning, such as cells that slow the player down (so would have a negative reinforcement), which could further display how the agent can find the best path.
 - The map could be built larger or scrolling to give a larger game world and interactivity.
 - The map is a fixed size and is larger than a small laptop screen. This may result in a user with a small laptop not being able to view the entire map in their viewport.
- BUGS
 - I have not run into any known crashes or performance issues when testing gameplay.